

Project 1, Session 2: Working with data and file structure

- Paired team review
- File structure and paths
- Principles for organizing data

Team discussion logistics

I'll provide a list of questions/tasks, including the ones described at the end of last class and a few related to the interpretation of your results.

For each question, one team will present:

- What you did, conceptually
- The code that you wrote, and how it achieved this goal
- How you checked your assumptions and verified that the code was doing what you intended

For conceptual parts/code outline/things to check, use the whiteboards!

For the team that is not presenting, this is your opportunity to ask questions and have a discussion. Is it clear what the presenters did? Did you approach the problem in a different way? Issues that were challenging for you, or solutions to problems that the presenters had?

After each question, the next team rotates to present. Some questions might focus more on

interpretation than writing code. In that case, presenters can focus on interpretation/results rather than code, per se.

Team discussion

Paired teams: 8-9, 6-7, 4-5, 1-2-3 (3 teams)

- How did you interpret the meaning of the dataset?
- Compute weighted scores for each variant
- Compute normalized scores for each variant, subtracting nonsense and normalizing by WT
- How did you treat multiple replicates in the above analyses?
- Which mutations appeared to be the most damaging? Which were most similar to WT?
- Were any results surprising? If so, how did you explain them?

The real starting point is different

Toy library	Published PTEN table
Counts per variant– replicate	Scores per variant
Median nonsense reference	Average nonsense reference
We calculate the score	We analyze the published score

What upstream information is absent from the score table?

About file systems

- A hierarchical “tree” of directories and files beginning at the **root** (/ or a drive letter)
- Each file has a *path* that tells the computer how to reach it
- File **extensions** hint at type, but the OS ultimately stores bytes

Home directories across operating systems

Platform	Typical home path	Example	Notes
macOS / Linux	/Users/<user> or /home/<user>	/Users/alice	Forward slash / separated
Windows	C:\Users\ <user>	C:\Users\Bob	Back-slash \ separated
HPC cluster	/home/<user>	/home/ab1234	Access SSH

Within WSL, main Windows folders are reached through `/mnt/c/<path>` , assuming the default c drive

Absolute vs relative paths

- **Absolute** – always starts from the root or drive: `/Users/alice/course-filesystem-lab/data/README.md`
- **Relative** – interpreted *from your current working directory*: `data/README.md`
- `.` = *this* directory `..` = *parent* directory

```
# Where am I? (print working directory)  
!pwd
```

```
%%bash  
# Doing the same thing with magic commands  
pwd
```

```
# Note that %% commands need to be at the top!  
%%bash  
pwd
```

```
# What is here? (list files, including hidden dotfiles)  
!ls -a
```

Command-line navigation

Action	Command	Handy options
Show location	<code>pwd</code>	—
List directory	<code>ls</code>	<code>-l</code> long, <code>-a</code> all, <code>-h</code> human-readable sizes
Change directory	<code>cd</code> <code><path></code>	<code>cd ~</code> , <code>cd ..</code>
Make directory	<code>mkdir</code> <code><name></code>	<code>-p</code> create parents
Copy / move	<code>cp</code> , <code>mv</code>	<code>-i</code> interactive prompt
Delete	<code>rm</code> , <code>rm -r</code>	<code>-i</code> , <i>never</i> <code>rm -rf /</code> 

```
%%bash
# Create an example project tree and explore it
mkdir -p demo_project/data demo_project/results
echo "example" > demo_project/data/dummy.txt
cd demo_project
pwd
echo "--- contents ---"
ls -R
cd ..
```

An example organized project folder

```
pten-project/  
  README.md  
  data/  
    raw/          # unchanged source  
    course/       # supplied teaching inputs  
    processed/    # team transformations  
  analysis.ipynb  
  outputs/  
    figures/  
    tables/
```

Keeping code, raw data, and generated results **separate** makes projects reproducible and easy to explore.

File paths in Python

A **relative path** starts from a chosen folder. An **absolute path** starts from the filesystem root.

```
from pathlib import Path
```

```
Path.cwd() # the kernel's current working directory
```

```
temp_path = Path.cwd() / 'instructor-notebook.ipynb'  
temp_path.exists()
```

```
temp_path.name, temp_path.suffix
```

File paths in Python

Prediction sheet A: What will we obtain when running the following expressions?

```
relative_path = Path("data") / "course" /  
"example.csv" # join paths with the / operator  
source_path = demo_root / relative_path  
source_path.exists(), source_path.name,  
source_path.suffix
```

Prediction sheet A: Would `relative_path` point to the same file if we started in a different directory? How does `demo_root` help to specify the path? In a project like this one, where would you keep an unchanged source data file or a calculated result?

Comma-separated values (CSV)

- Entries separated by commas
- Rows separated by new line `\n` characters
- The first row gives the names of the variables

Reading a file into pandas

```
import pandas as pd

example = pd.read_csv(source_path)
example
```

Pandas uses NaN to represent missing data.

Prediction sheet B: How many entries are missing for each column? Could NaN be interpreted as zero?

```
example[["score", "replicate_1",  
"replicate_2"]].isna().sum()
```

Could another person interpret your table?

State what **one row** represents.

Keep identifiers, variables, units, and missing-value meanings explicit.

Would a category recorded only as a cell color survive saving a CSV?

PTEN data: trace a label to the reference

p.Met1Val → reference Met → position 1 → alternative Val

What Python index checks the reference amino acid?

What should happen to a stop label or an out-of-range position?

Assignment for next time

With your team, examine the PTEN scores and reference sequence file (MaveDB

urn:mavedb:00000013-a-1):

- Explain what each row and column means in the score tables
- Parse mutation labels in the score tables to identify the site, WT residue, and mutant, and verify that the listed WT residue matches the reference
- Compare score distributions for WT, missense, nonsense, and synonymous mutations

For the beginning of class, be ready to

- Describe file locations with Python paths
- Select data subsets, get missing values, and add new columns in pandas